

LLM ফাইন-টিউনিং বাংলা গাইড

Large Language Model Fine-Tuning

বেসিক থেকে অ্যাডভান্স

১০টি অধ্যায়

বাংলা মাধ্যম

Python + PyTorch

সংস্করণ ১.০ · ২০২৫

Claude AI কর্তৃক প্রস্তুতকৃত

সূচিপত্র

১	LLM কী ? প্রাথমিক ধারণা	বেসিক
২	ফাইন-টিউনিং কী এবং কেন ?	বেসিক
৩	ডেটা প্রস্তুতি — Dataset collection ও formatting	বেসিক
৪	SFT — Supervised Fine-Tuning	ইন্টারমিডিয়েট
৫	LoRA ও PEFT — কম মেমরিতে ফাইন-টিউনিং	ইন্টারমিডিয়েট
৬	RLHF ও DPO — মানুষের পছন্দ থেকে শেখা	ইন্টারমিডিয়েট
৭	মডেল মূল্যায়ন — Evaluation metrics ও benchmarks	ইন্টারমিডিয়েট
৮	ডিপ্লয়মেন্ট ও সার্ভিং — প্রোডাকশনে মডেল চালানো	অ্যাডভান্স
৯	অ্যাডভান্স কৌশল — Merging, RAG, Unsloth	অ্যাডভান্স
১০	বাস্তব প্রজেক্ট — বাংলা চ্যাটবট তৈরির পূর্ণ গাইড	অ্যাডভান্স

LLM কী ? প্রাথমিক ধারণা

Large Language Models পরিচিতি · বেসিক

LLM কী ?

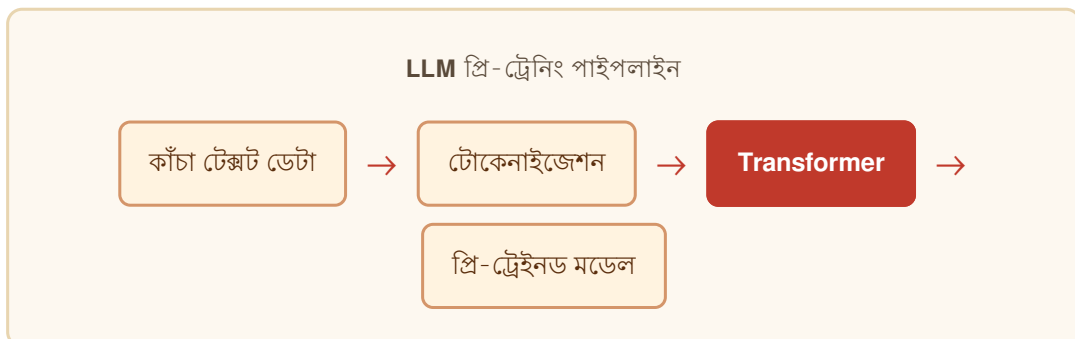
Large Language Model (LLM) হলো এক ধরনের কৃত্রিম বুদ্ধিমত্তার মডেল যা বিশাল পরিমাণ টেক্সট ডেটা দিয়ে ট্রেন করা হয়। এই মডেলগুলো মানুষের মতো ভাষা বুঝতে, তৈরি করতে এবং অনুবাদ করতে পারে।

- **GPT** সিরিজ: OpenAI-এর তৈরি মডেল (GPT-4, GPT-4o)
- **Claude**: Anthropic-এর তৈরি মডেল
- **LLaMA**: Meta-এর ওপেন-সোর্স মডেল
- **Gemini**: Google DeepMind-এর মডেল
- **Mistral**: ইউরোপীয় ওপেন-সোর্স মডেল

মূল ধারণা

LLM মূলত Transformer আর্কিটেকচারের উপর ভিত্তি করে তৈরি। ২০১৭ সালে Google-এর "Attention is All You Need" পেপারে এই আর্কিটেকচার প্রকাশিত হয়েছিল।

LLM কীভাবে কাজ করে ?



- **টোকেনাইজেশন**: টেক্সটকে ছোট ছোট "টোকেন"-এ ভাগ করা হয়
- **Embedding**: প্রতিটি টোকেনকে সংখ্যার ভেক্টরে রূপান্তর করা হয়
- **Self-Attention**: শব্দগুলোর মধ্যে সম্পর্ক বোঝা হয়

- **Next Token Prediction:** পরবর্তী শব্দ কী হবে তা অনুমান করা হয়

175B

GPT-3 প্যারামিটার

70B

LLaMA-3 প্যারামিটার

~ 1T

টোকেন প্রি-ট্রেনিং

96+

Transformer লেয়ার

ফাইন-টিউনিং কী এবং কেন ?

Pre-training vs Fine-tuning · বেসিক

ফাইন-টিউনিং কী ?

একটি প্রি-ট্রেন্ড LLM-কে নির্দিষ্ট কাজের জন্য আরও ভালো করে তোলার প্রক্রিয়াই হলো ফাইন-টিউনিং। অর্থাৎ, বিশাল ডেটায় শেখা মডেলকে আমাদের নির্দিষ্ট ডোমেইনের ছোট ডেটায় আবার ট্রেন করা হয়।

প্রি-ট্রেনিং

- ▶ কোটি কোটি ডকুমেন্ট দরকার
- ▶ মাসের পর মাস সময়
- ▶ লক্ষ লক্ষ ডলার খরচ
- ▶ সাধারণ জ্ঞান শেখে
- ▶ হাজার GPU দরকার

ফাইন-টিউনিং (প্রস্তাবিত)

- ▶ কয়েক হাজার উদাহরণ যথেষ্ট
- ▶ ঘণ্টা থেকে কয়েকদিন
- ▶ তুলনামূলক কম খরচ
- ▶ নির্দিষ্ট কাজে দক্ষ
- ▶ একটি GPU-তেও সম্ভব

কেন ফাইন-টিউনিং করবেন ?

- **ডোমেইন-নির্দিষ্টতা:** মেডিকেল, আইনি, বা বাংলা ভাষার ডেটায় বিশেষ দক্ষতা অর্জন
- **ইনস্ট্রাকশন ফলো:** নির্দিষ্ট ফরম্যাটে উত্তর দেওয়া শেখানো
- **গোপনীয়তা:** নিজের সার্ভারে মডেল চালানো, ডেটা বাইরে যাবে না
- **খরচ কমানো:** ছোট ফাইন-টিউনড মডেল, বড় API-এর চেয়ে সস্তা
- **অফলাইন ব্যবহার:** ইন্টারনেট ছাড়া চালানো সম্ভব

উদাহরণ ব্যবহার ক্ষেত্র

বাংলাদেশের একটি হাসপাতাল তাদের মেডিকেল রিপোর্ট বাংলায় সারাংশ তৈরি করতে LLaMA-3 ফাইন-টিউন করতে পারে — শুধুমাত্র ১০,০০০ রিপোর্টের ডেটা ব্যবহার করে।

ফাইন-টিউনিং-এর ধরন

- **Supervised Fine-Tuning (SFT):** ইনপুট-আউটপুট জোড়া দিয়ে শেখানো — সবচেয়ে সহজ
- **RLHF:** মানুষের ফিডব্যাক থেকে Reinforcement Learning
- **PEFT / LoRA:** কম প্যারামিটার আপডেট করে দক্ষ ফাইন-টিউনিং
- **Instruction Tuning:** নির্দেশনা মানতে শেখানো
- **DPO:** পছন্দ-অপছন্দ থেকে শেখা (Direct Preference Optimization)

ডেটা প্রস্তুতি

Dataset collection, cleaning এবং formatting · বেসিক

ডেটাসেট তৈরি — সবচেয়ে গুরুত্বপূর্ণ ধাপ

ফাইন-টিউনিং-এর সাফল্য ৭০%-ই নির্ভর করে ডেটার গুণগত মানের উপর। "Garbage in, Garbage out" — খারাপ ডেটা দিলে খারাপ মডেল পাবেন।

ডেটার ফরম্যাট

সবচেয়ে প্রচলিত ফরম্যাট হলো Instruction-Input-Output বা Conversation ফরম্যাট:

```
dataset_example.jsonl

// Instruction format (Alpaca style)
{
  "instruction": "নিচের বাংলা টেক্সটটি ইংরেজিতে অনুবাদ করুন",
  "input": "আমি বাংলাদেশে থাকি।",
  "output": "I live in Bangladesh."
}

// Conversation format (ChatML style)
{
  "messages": [
    {"role": "system", "content": "আপনি একজন সহায়ক AI"},
    {"role": "user", "content": "পাইথন কী?"},
    {"role": "assistant", "content": "পাইথন একটি প্রোগ্রামিং ভাষা..."}
  ]
}
```

ডেটা পরীক্ষার করা

```
data_cleaning.py

import json
from datasets import Dataset

def clean_data(examples):
    cleaned = []
    for ex in examples:
        # খুব ছোট বা বড় উদাহরণ বাদ দিন
        if 10 < len(ex['output']) < 2000:
            cleaned.append(ex)
    return cleaned

# ডেটা লোড করুন
with open('data.jsonl') as f:
    data = [json.loads(line) for line in f]

clean = clean_data(data)
dataset = Dataset.from_list(clean)
print(f"মোট উদাহরণ: {len(dataset)}")
```

কতটুকু ডেটা দরকার ?

সাধারণ কাজের জন্য ১,০০০–১০,০০০ উদাহরণ। জটিল ডোমেইনের জন্য ৫০,০০০+।
গুণমান সংখ্যার চেয়ে বেশি গুরুত্বপূর্ণ।

বাংলা ডেটাসেট সোর্স

- **IndicCorp**: ভারতীয় উপমহাদেশের ভাষার বড় কর্পাস
- **CC-100**: Common Crawl থেকে বাংলা ডেটা সংগ্রহ
- **BanglaLM**: Hugging Face-এ উপলব্ধ বাংলা ডেটাসেট
- **নিজের ডেটা**: প্রশ্নোত্তর, রিভিউ, নিউজ আর্টিকেল সংগ্রহ করুন

SFT — Supervised Fine-Tuning

মূল ফাইন-টিউনিং পদ্ধতি · ইন্টারমিডিয়েট

SFT কী ?

Supervised Fine-Tuning হলো সবচেয়ে সরল এবং প্রচলিত ফাইন-টিউনিং পদ্ধতি। এখানে আমরা মডেলকে (input, expected_output) জোড়া দেখিয়ে শেখাই। মডেল শেখে — এই ধরনের প্রশ্নে এই ধরনের উত্তর দিতে হয়।

Hugging Face দিয়ে SFT

```
sft_training.py

from transformers import AutoModelForCausalLM, AutoTokenizer
from trl import SFTTrainer, SFTConfig
from datasets import load_dataset

# মডেল ও টোকেনাইজার লোড
model_name = "meta-llama/Llama-3.2-1B"
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

# ডেটাসেট লোড
dataset = load_dataset("আপনার_ডেটাসেট", split="train")

# ট্রেনিং কনফিগ
config = SFTConfig(
    output_dir="./finetuned_model",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    learning_rate=2e-5,
    max_seq_length=512,
    logging_steps=10,
)

# ট্রেনার তৈরি ও ট্রেনিং শুরু
trainer = SFTTrainer(
    model=model,
    args=config,
    train_dataset=dataset,
    tokenizer=tokenizer,
)
trainer.train()
trainer.save_model()
```

গুরুত্বপূর্ণ হাইপারপ্যারামিটার

- **Learning Rate (lr):** সাধারণত $1e-5$ থেকে $5e-5$ — LLM ফাইন-টিউনিং-এ খুব গুরুত্বপূর্ণ
- **Batch Size:** GPU মেমরি অনুযায়ী, সাধারণত ৪–৩২
- **Epochs:** ১–৫ — বেশি হলে Overfitting হতে পারে
- **Max Sequence Length:** ৫১২–২০৪৮ টোকেন

- **Warmup Steps:** শুরুতে learning rate ধীরে বাড়ানো হয়

সতর্কতা: **CATASTROPHIC FORGETTING**

ফাইন-টিউনিং করলে মডেল পুরনো জ্ঞান ভুলে যেতে পারে। এটি "Catastrophic Forgetting" সমস্যা। সমাধান: LoRA বা খুব কম learning rate ব্যবহার করুন।

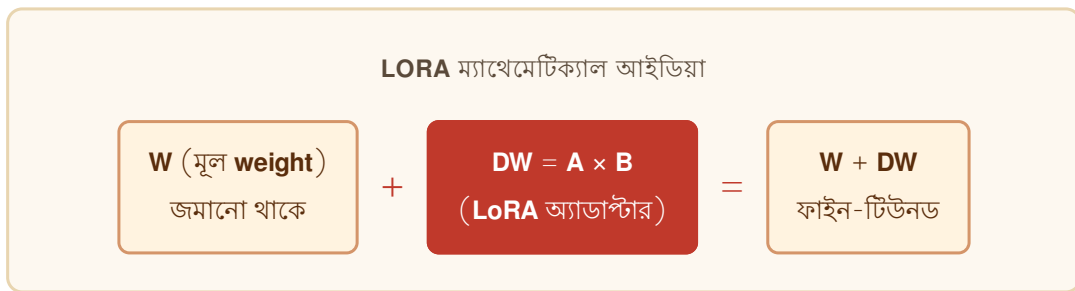
LoRA ও PEFT পদ্ধতি

Parameter-Efficient Fine-Tuning · ইন্টারমিডিয়েট

LoRA কী ? কেন প্রয়োজন ?

সম্পূর্ণ LLM ফাইন-টিউন করতে বিশাল GPU মেমরি লাগে। LoRA (Low-Rank Adaptation) এই সমস্যার সমাধান — মডেলের মাত্র ০.১–১% প্যারামিটার আপডেট করে একই মানের ফাইন-টিউনিং সম্ভব !

LoRA কীভাবে কাজ করে ?



মূল weight matrix W (যেমন 1000×1000) ফ্রিজ থাকে। শুধু দুটো ছোট matrix A এবং B শেখা হয় — এতে প্যারামিটার সংখ্যা ৯৯% কমে !

QLoRA — আরও কম মেমরিতে

```
qlora_setup.py

from peft import LoraConfig, get_peft_model
from transformers import BitsAndBytesConfig
import torch

# 8-বিট কোয়ান্টাইজেশন কনফিগ
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16
)

# LoRA কনফিগ
lora_config = LoraConfig(
    r=16, # rank - কত ছোট matrix
    lora_alpha=32, # scaling factor
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable params: 4,194,304 || all params: 1.24B (0.34%)
```

0.3%

LoRA ট্রেনেবল
প্যারামিটার

4-bit

QLoRA মেমরি

~6GB

7B মডেল QLoRA-
তে

~ 100%

মান বজায় রাখে

র‍্যাঙ্ক (**R**) কতটুকু রাখবেন ?

সাধারণ কাজে $r=8$ বা $r=16$ যথেষ্ট। জটিল কাজে $r=32$ বা $r=64$ ব্যবহার করুন। বড় r মানে বেশি প্যারামিটার ও ভালো মান কিন্তু বেশি মেমরি।

RLHF ও DPO

মানুষের পছন্দ থেকে শেখা · ইন্টারমিডিয়েট

RLHF কী ?

Reinforcement Learning from Human Feedback (RLHF) হলো GPT-4, Claude এবং ChatGPT-কে "helpful, harmless, honest" করার মূল পদ্ধতি। এখানে মানুষ মডেলের দুটো উত্তর দেখে ভালো উত্তরটি বেছে নেয় — এই তথ্য দিয়ে মডেল শেখে।

RLHF পাইপলাইন



- ধাপ ১: প্রথমে SFT দিয়ে বেস মডেল তৈরি করা হয়
- ধাপ ২: মানুষের রেটিং থেকে "Reward Model" শেখানো হয়
- ধাপ ৩: PPO অ্যালগরিদম দিয়ে মডেলকে বেশি reward পেতে অপ্টিমাইজ করা হয়

DPO — RLHF-এর সহজ বিকল্প

Direct Preference Optimization (DPO) RLHF-এর জটিলতা এড়িয়ে সরাসরি "chosen vs rejected" উত্তর জোড়া থেকে শেখে। ২০২৩ সালে প্রকাশিত এই পদ্ধতি এখন খুব জনপ্রিয়।

```
dpo_training.py

from trl import DPOTrainer, DPOConfig

# DPO ডেটাসেট ফরম্যাট:
# {"prompt": "প্রশ্ন", "chosen": "ভালো উত্তর", "rejected": "খারাপ উত্তর"}

dpo_config = DPOConfig(
    output_dir="./dpo_model",
    num_train_epochs=1,
    per_device_train_batch_size=2,
    beta=0.1,          # KL penalty coefficient
    learning_rate=5e-7,
)

trainer = DPOTrainer(
    model=sft_model,
    ref_model=ref_model,
    args=dpo_config,
    train_dataset=preference_dataset,
    tokenizer=tokenizer,
)
trainer.train()
```

RLHF

- ▶ ৩টি পৃথক মডেল দরকার
- ▶ PPO অ্যালগরিদম জটিল
- ▶ ট্রেনিং অস্থির হতে পারে
- ▶ বেশি মেমরি লাগে

DPO (প্রস্তাবিত)

- ▶ Reward model দরকার নেই
- ▶ সরল ও স্থিতিশীল
- ▶ কম মেমরি ব্যবহার
- ▶ ভালো পারফরম্যান্স

মডেল মূল্যায়ন

Evaluation metrics ও benchmarks · ইন্টারমিডিয়েট

কীভাবে বুঝবেন মডেল ভালো হয়েছে ?

ফাইন-টিউনিং-এর পরে মডেলের মান মাপতে বিভিন্ন মেট্রিক ও পদ্ধতি ব্যবহার করা হয়।

স্বয়ংক্রিয় মেট্রিক

- **Perplexity:** মডেল কতটা "বিস্মিত" — কম মানে ভালো পারফরম্যান্স
- **BLEU Score:** অনুবাদ মূল্যায়নে ব্যবহৃত, রেফারেন্সের সাথে মিল দেখে
- **ROUGE:** সারাংশ মূল্যায়নে, recall-ভিত্তিক পরিমাপ
- **BERTScore:** অর্থগত মিল পরিমাপ করে
- **Exact Match:** প্রশ্নোত্তরে হুবহু মিল পরিমাপ

LLM-as-Judge পদ্ধতি

```
llm_evaluation.py

from anthropic import Anthropic

client = Anthropic()

def evaluate_with_llm(question, model_answer, reference):
    prompt = f"""আপনি একজন নিরপেক্ষ মূল্যায়নকারী।
    প্রশ্ন: {question}
    মডেলের উত্তর: {model_answer}
    সঠিক উত্তর: {reference}

    ১-৫ স্কেলে উত্তরের মান দিন এবং কারণ বলুন।"""

    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text
```

বেঞ্চমার্ক ডেটাসেট

- **MMLU**: বহু বিষয়ের জ্ঞান পরীক্ষা (৫৭টি ক্যাটেগরি)
- **HumanEval**: কোড জেনারেশন মূল্যায়ন
- **MT-Bench**: মাল্টি-টার্ন কথোপকথন মূল্যায়ন
- **BanglaLM Bench**: বাংলা ভাষার মূল্যায়ন বেঞ্চমার্ক

সেরা পদ্ধতি

স্বয়ংক্রিয় মেট্রিকের পাশাপাশি ২০-৫০টি উদাহরণ নিজে হাতে পরীক্ষা করুন। Human evaluation সবচেয়ে বিশ্বাসযোগ্য মূল্যায়ন পদ্ধতি।

ডিপ্লয়মেন্ট ও সার্ভিং

প্রোডাকশনে মডেল চালানো · অ্যাডভান্স

ফাইন-টিউনড মডেল সার্ভ করার পদ্ধতি

মডেল ট্রেন করার পরে সেটি রিয়েল অ্যাপ্লিকেশনে ব্যবহার করতে ডিপ্লয়মেন্ট দরকার। বিভিন্ন পদ্ধতি রয়েছে গতি ও খরচের ভারসাম্য রেখে।

vLLM (প্রস্তাবিত)

- ▶ সবচেয়ে দ্রুত ইনফারেন্স
- ▶ Continuous batching সাপোর্ট
- ▶ OpenAI-সামঞ্জস্যপূর্ণ API
- ▶ GPU দরকার হয়

Ollama (সহজ)

- ▶ লোকাল কম্পিউটারে চলে
- ▶ CPU-তেও চালানো যায়
- ▶ সহজ ইনস্টলেশন
- ▶ তুলনামূলক ধীরে চলে

vLLM দিয়ে সার্ভার চালানো

```
deploy_vllm.sh + client.py

# সার্ভার চালু করুন
$ python -m vllm.entrypoints.openai.api_server \
  --model ./finetuned_model \
  --port 8000

# Python থেকে ব্যবহার
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1", api_key="na")

response = client.chat.completions.create(
    model="finetuned_model",
    messages=[
        {"role": "user", "content": "বাংলায় কিছু বলুন"}
    ]
)
print(response.choices[0].message.content)
```

মডেল কোয়ান্টাইজেশন (প্রোডাকশনে)

- **GPTQ**: পোস্ট-ট্রেনিং কোয়ান্টাইজেশন, GPU-তে দ্রুত চলে
- **GGUF/llama.cpp**: CPU-তে চালানোর জন্য সর্বোত্তম
- **AWQ**: অ্যাঙ্কিভেশন-সচেতন কোয়ান্টাইজেশন পদ্ধতি
- **INT8/INT4**: মেমরি ও গতির মধ্যে আপোষ করে

পারফরম্যান্স টিপস

প্রোডাকশনে Flash Attention 2 ব্যবহার করুন (৩-৫x গতি বৃদ্ধি পাবেন)। Tensor Parallelism দিয়ে একাধিক GPU-তে ভাগ করুন। KV cache-এর আকার বাড়িয়ে throughput উন্নত করুন।

অ্যাডভান্স কৌশল

Merging, RAG, Unsloth, Flash Attention · অ্যাডভান্স

মডেল মার্জিং

একাধিক ফাইন-টিউনড মডেলকে একত্রিত করে একটি শক্তিশালী মডেল তৈরি করা সম্ভব — এটি "Model Merging"।

```
merge_config.yaml (mergekit)

models:
  - model: bangla-finetuned-v1
    parameters: {weight: 0.6}
  - model: code-finetuned-v1
    parameters: {weight: 0.4}
merge_method: ties
base_model: meta-llama/Llama-3.2-1B

# মার্জ করুন
$ mergekit-yaml merge_config.yaml ./merged_model
```

Unsloth — দ্রুততর ফাইন-টিউনিং

```
unsloth_training.py

from unsloth import FastLanguageModel

# ২x দ্রুত লোড ও ট্রেনিং, ৮০% কম মেমরি
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/llama-3.2-1b-bnb-4bit",
    max_seq_length=2048,
    load_in_4bit=True,
)
model = FastLanguageModel.get_peft_model(
    model, r=16, lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
)
```

RAG vs Fine-tuning — কোনটি বেছে নেবেন ?

RAG ব্যবহার করুন যখন

- ▶ ডেটা ঘন ঘন আপডেট হয়
- ▶ সোর্স উল্লেখ করতে হবে
- ▶ নির্দিষ্ট ডকুমেন্ট থেকে উত্তর
- ▶ দ্রুত শুরু করতে চান

Fine-tuning করুন যখন

- ▶ মডেলের আচরণ বদলাতে হবে
- ▶ নির্দিষ্ট ফরম্যাট দরকার
- ▶ ডোমেইন-নির্দিষ্ট ভাষা শেখাতে হবে
- ▶ লেটেন্সি কম চাই

সেরা কৌশল: **RAG + FINE-TUNING** একসাথে

অনেক প্রোডাকশন সিস্টেম দুটোই ব্যবহার করে — ফাইন-টিউনিং করা বেস মডেলে RAG যোগ করলে সর্বোচ্চ পারফরম্যান্স পাওয়া যায়।

বাস্তব প্রজেক্ট

বাংলা চ্যাটবট তৈরির পূর্ণ গাইড · অ্যাডভান্স

প্রজেক্ট: বাংলা কাস্টমার সাপোর্ট চ্যাটবট

এই প্রজেক্টে আমরা LLaMA-3.2-1B মডেলকে বাংলায় ই-কমার্স কাস্টমার সাপোর্টের জন্য ফাইন-টিউন করব।

পূর্ণ পাইপলাইন




```

full_project.py

# ধাপ ১: প্রয়োজনীয় লাইব্রেরি ইনস্টল
# pip install unsloth trl transformers datasets peft

from unsloth import FastLanguageModel
from trl import SFTTrainer, SFTConfig
from datasets import Dataset
import json

# ধাপ ২: মডেল লোড
model, tokenizer = FastLanguageModel.from_pretrained(
    "unsloth/llama-3.2-1b-bnb-4bit",
    max_seq_length=1024,
    load_in_4bit=True
)
model = FastLanguageModel.get_peft_model(
    model, r=16, lora_alpha=32
)

# ধাপ ৩: বাংলা ডেটা ফরম্যাট করুন
def format_prompt(example):
    return {"text": f"<|system|>
আপনি একটি বাংলাদেশি ই-কমার্স সাইটের সহায়ক।
<|user|>
{example['question']}
<|assistant|>
{example['answer']}<|end|>"}

# ধাপ ৪: ট্রেনিং শুরু করুন
trainer = SFTTrainer(
    model=model,
    args=SFTConfig(
        output_dir="./bangla_chatbot",
        num_train_epochs=3,
        per_device_train_batch_size=4,
        learning_rate=2e-4,
        dataset_text_field="text",
    ),
    train_dataset=dataset.map(format_prompt),
    tokenizer=tokenizer,
)
trainer.train()

# ধাপ ৫: GGUF ফরম্যাটে সেভ করুন (ollama-র জন্য)
model.save_pretrained_gguf(
    "bangla_chatbot_gguf",
    tokenizer,

```



```
quantization_method="q4_k_m"  
)
```

পরবর্তী শেখার পথ

- **Multimodal Fine-tuning:** ছবি + টেক্সট একসাথে ফাইন-টিউন করা
- **Constitutional AI:** নীতিমালা থেকে মডেল শেখানো
- **Mixture of Experts (MoE):** বড় মডেল আর্কিটেকচার বোঝা
- **Continual Learning:** ক্রমাগত নতুন তথ্য শেখানো
- **Hugging Face Hub:** মডেল শেয়ার করুন সমাজের সাথে

অভিনন্দন! আপনি সম্পূর্ণ গাইড শেষ করেছেন

এখন আপনার কাছে LLM ফাইন-টিউনিং-এর সম্পূর্ণ জ্ঞান আছে। প্র্যাকটিস করুন, পরীক্ষা করুন, এবং বাংলায় AI তৈরিতে অবদান রাখুন!